

ArachneX

AI-Native XSS Discovery, Detection,
Exploitation & Reporting Framework

User & Operator Guide

Installation · Configuration · Scanning · Reporting · Tuning

by Rhishinathvarma Marimuthu

Version 0.1.0 · June 2026

Licensed under MIT · for authorized security testing only

Contents

Contents	3
1 · About ArachneX	6
1.1 · What you can do with it	6
1.2 · What this guide covers	6
2 · Quickstart (90 seconds)	7
3 · Prerequisites	8
3.1 · System requirements	8
3.2 · Required Python packages	8
4 · Installation	9
4.1 · Standard installation	9
4.2 · Development installation (with tests)	9
4.3 · Optional: ML extras	9
4.4 · Verifying your install	9
5 · Configuration	10
5.1 · Environment variables	10
5.2 · The .env file	10
5.3 · Cookies and authentication headers	10
5.4 · Proxying through Burp / mitmproxy	10
6 · Your first scan	12
6.1 · Start the demo target	12
6.2 · Run the scan	12
6.3 · What you will see	12
6.4 · Anatomy of a finding	12
7 · CLI reference	13
7.1 · arachnex scan	13
7.2 · arachnex dom	13
7.3 · arachnex blind	14
7.4 · arachnex version	14
8 · Scan modes	15
8.1 · quick	15
8.2 · standard	15
8.3 · deep	15

8.4 · bug-bounty	15
9 · Understanding reports	16
9.1 · HTML report	16
9.2 · JSON report	16
9.3 · Markdown report	16
9.4 · Severity and CVSS	16
10 · The headless oracle	17
10.1 · How it works	17
10.2 · Headless modes	17
10.3 · Performance	17
11 · Blind XSS infrastructure	18
11.1 · The collaborator server	18
11.2 · Deploying with TLS	18
11.3 · Payload variants delivered	18
11.4 · Reviewing callbacks	18
12 · AI features	20
12.1 · LLM payload generation	20
12.2 · Genetic-algorithm payload evolution	20
12.3 · Auditable triage scoring	20
12.4 · LLM cost estimation	20
13 · Recipes	21
13.1 · Authenticated SPA on a known host	21
13.2 · API endpoint accepting JSON	21
13.3 · A scope file of multiple hosts	21
13.4 · A CI gate that fails on High/Critical findings	21
13.5 · Investigating a specific suspected parameter	21
13.6 · Static-only DOM-XSS audit (no traffic)	22
13.7 · Routing through Burp Suite	22
13.8 · Blind XSS campaign	22
14 · Troubleshooting	23
14.1 · "playwright: command not found"	23
14.2 · "Executable doesn't exist at chromium ..."	23
14.3 · SSL: CERTIFICATE_VERIFY_FAILED	23
14.4 · 429 Too Many Requests	23
14.5 · LLM payload generation seems disabled	23

- 14.6 · Scan is too slow 23
- 14.7 · "could not start headless oracle" 24
- 14.8 · A finding shows reflection but no execution 24
- 15 · Performance tuning 25**
 - 15.1 · --threads 25
 - 15.2 · --rps 25
 - 15.3 · The headless pool 25
 - 15.4 · A representative budget 25
- 16 · Ethics & legal 26**
 - 16.1 · Jurisdictions 26
 - 16.2 · What "authorization" looks like 26
 - 16.3 · Bug bounty etiquette 26
- 17 · Glossary 27**
- Appendix A · Environment variable reference 28**
- Appendix B · Project directory layout 28**

1 · About ArachneX

ArachneX is a Python framework for finding, verifying, and reporting cross-site scripting (XSS) vulnerabilities. Unlike static-payload fuzzers, it classifies each reflection into one of seventeen syntactic contexts, generates payloads grounded in the actual byte neighbourhood around the reflection, evolves payloads against the defending WAF using a genetic algorithm, and confirms execution in a real headless browser before reporting anything.

1.1 · What you can do with it

- Scan a single URL, a scope file of hosts, or a crawled application.
- Detect reflected, DOM-based, mutation (mXSS), stored, and blind XSS.
- Verify findings in real Chromium via Playwright, with three independent execution signals.
- Evolve payloads against modern WAFs (Cloudflare, AWS WAF, Akamai, others).
- Receive blind-XSS callbacks via an embedded FastAPI collaborator server.
- Export findings as HTML (self-contained), JSON (schema-versioned), or Markdown.
- Score every finding with a CVSS 4.0 vector and a human-readable rationale.

1.2 · What this guide covers

This document walks you from installation through your first scan, through every command-line option, and through advanced topics like blind-XSS infrastructure, genetic-algorithm tuning, and the headless oracle. The final chapters cover troubleshooting, performance, ethics, and a glossary.

DANGER · Authorization required

ArachneX is offensive tooling. Run it only against systems you own or are explicitly authorized in writing to test. Unauthorized scanning is criminal under the IT Act 2000 (India), CFAA (USA), Computer Misuse Act 1990 (UK), and equivalent laws elsewhere. The authors accept no liability for misuse.

2 · Quickstart (90 seconds)

Five commands. If you already have Python 3.11+ and pip, you can have your first verified XSS finding in under two minutes.

Step 1 — Unpack and enter the project

```
unzip arachnex.zip
cd arachnex
```

Step 2 — Install

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e .
playwright install chromium
```

Step 3 — Run the deliberately-vulnerable demo target

Open a second terminal so you can keep the target running:

```
# in terminal 2
pip install flask
python examples/demo_target.py
```

You should see ArachneX demo target — <http://127.0.0.1:5050/> in terminal 2.

Step 4 — Scan it

```
# back in terminal 1
arachnex scan -u http://127.0.0.1:5050/ \
  --mode deep \
  --format html --format json --format md
```

Step 5 — Open the report

```
open arachnex-out/<scan_id>/report.html # macOS
xdg-open arachnex-out/<scan_id>/report.html # Linux
```

You will see between five and ten verified findings, each with the payload that worked, a captured screenshot, the reflection context that was classified, and a CVSS 4.0 vector. If that all works, your installation is healthy and you are ready for real targets.

3 · Prerequisites

3.1 · System requirements

Option	Description
Python 3.11+	ArachneX uses asyncio TaskGroup, structural-pattern matching, and PEP 604 unions — none of which are available on 3.10.
RAM 4 GB minimum	Chromium contexts cost around 80 MB each. Eight concurrent headless contexts plus the Python heap fit in 2 GB; we recommend 4 GB as headroom.
Disk 1 GB	Most of this is Playwright's Chromium download (~300 MB) and scan output (screenshots, raw response snippets).
OS Linux/macOS	Tested on Ubuntu 22.04+, macOS 13+. Windows works under WSL2; native Windows is best-effort.
Network	Outbound HTTPS to the target. Anthropic API access if you want LLM-generated payloads (optional).

3.2 · Required Python packages

Everything is declared in `pyproject.toml`. The runtime depends on:

- **httpx[http2]**, **tenacity** — async HTTP client with retries
- **beautifulsoup4**, **lxml**, **html5lib**, **esprima** — parsers
- **playwright** — headless Chromium driver
- **pydantic 2.x**, **pydantic-settings** — typed configuration
- **typer**, **rich** — CLI
- **jinja2**, **orjson** — reporting
- **anthropic** — optional, for LLM payload generation
- **fastapi**, **uvicorn** — only required when running the blind collab server

NOTE · What if Playwright fails to install?

On minimal Linux images, Playwright's Chromium needs additional shared libraries. Run `playwright install-deps chromium` after the main install. On Debian/Ubuntu this resolves missing `libnss`, `libatk`, `libdrm`, and similar dependencies.

5 · Configuration

ArachneX accepts configuration through three layers, in order of precedence (highest wins): command-line flags, environment variables, a `.env` file in the working directory.

5.1 · Environment variables

Option	Description
ANTHROPIC_API_KEY	Required for LLM-generated payloads. Without it, ArachneX still scans using the static library plus genetic-algorithm evolution; LLM features are silently disabled.
ARX_LLM_MODEL	Model name. Defaults to <i>claude-sonnet-4-6</i> .
ARX_OUTPUT_DIR	Default report output directory if <code>--out</code> is not given.
ARX_LOG_LEVEL	DEBUG / INFO / WARNING / ERROR. Default INFO.
ARX_USER_AGENT	Override the default User-Agent. Defaults to ArachneX/0.1.0.
ARX_PROXY	HTTP proxy URL applied to all outgoing requests.

5.2 · The `.env` file

Drop this in the project root before running:

```
ANTHROPIC_API_KEY=sk-ant-...
ARX_LLM_MODEL=claude-sonnet-4-6
ARX_OUTPUT_DIR=./arachnex-out
ARX_LOG_LEVEL=INFO
```

5.3 · Cookies and authentication headers

Pass authenticated session cookies and any required headers via `--cookie` and `-H` flags. Both can be repeated. ArachneX never logs in for you — it expects to be handed a session that is already authenticated.

```
arachnex scan -u https://app.example.com/dashboard \
  --cookie 'session=eyJhbGciOi...' \
  --cookie 'csrftoken=abc123' \
  -H "Authorization: Bearer eyJh..." \
  -H "X-Forwarded-For: 127.0.0.1"
```

5.4 · Proxying through Burp / mitmproxy

For traffic inspection and debugging, route ArachneX through your local intercepting proxy:

```
arachnex scan -u https://target/ \
  --proxy http://127.0.0.1:8080
```

WARNING · TLS verification

When proxying through Burp, you will hit TLS verification errors unless Burp's CA is installed in your Python certificate bundle. Either install the Burp CA into your system trust store, or set the environment variable `ARX_VERIFY_TLS=0` to disable verification for the scan. Never disable verification for production scans.

6 · Your first scan

We will run a full scan against the bundled vulnerable demo target. Every endpoint of `examples/demo_target.py` is deliberately exploitable, so you should see findings across most detection classes.

6.1 · Start the demo target

```
pip install flask # one-time
python examples/demo_target.py
```

The demo exposes ten endpoints, each illustrating a different reflection context (HTML body, double-quoted attribute, event handler, JS string, SVG, URL href, mXSS via `<listing>`, POST body, DOM-XSS via `location.hash`, and a stored/blind comment endpoint).

6.2 · Run the scan

```
arachnex scan -u http://127.0.0.1:5050/ \
  --mode deep \
  --threads 25 --rps 20 \
  --format html --format json --format md \
  --out ./arachnex-out
```

6.3 · What you will see

ArachneX prints a banner with effective configuration, then streams discovery progress, then per-finding emit lines in real time. A typical successful finding line looks like:

```
◆ HIGH  reflected XSS in `q` — http://127.0.0.1:5050/reflect/svg
      payload=<svg onload=alert(31337)>
```

At the end of the scan, the report locations are printed. Open `report.html` — every finding shows the exact payload, the classified context, the response snippet with the reflection highlighted, and a screenshot of the executing page.

6.4 · Anatomy of a finding

Each finding card contains:

- **Severity badge:** Critical, High, Medium, Low, or Info.
- **Confidence:** 0.00 to 1.00, from the triage classifier.
- **CVSS 4.0 vector:** machine-readable, plus a calibrated numeric score.
- **Class:** reflected, stored, blind, DOM, or mXSS.
- **Injection point:** parameter name and location (query, body, header, cookie, fragment).
- **Payload:** the exact bytes that succeeded.
- **Payload lineage:** the chain of transforms applied during GA evolution (if any).
- **Response snippet:** ±100 characters around the reflection, with the canary highlighted.
- **Browser proof:** console output, captured cookies, and an inline screenshot.
- **WAF signal:** the detected vendor and confidence (if any was detected).

7 · CLI reference

ArachneX exposes four subcommands. Run `arachnex --help` for a top-level summary, or `arachnex SUBCOMMAND --help` for per-command help.

7.1 · arachnex scan

Run a full scan. The main command you will use.

Option	Description
<code>-u, --url URL</code>	Target URL. Repeat for multiple seeds.
<code>-s, --scope FILE</code>	Newline-delimited URL/host scope file.
<code>--exclude HOST</code>	Out-of-scope host. Repeat as needed.
<code>-m, --mode MODE</code>	quick standard deep bug-bounty. Default: standard.
<code>--headless MODE</code>	off verify always. Default: verify (verify findings only).
<code>-t, --threads N</code>	Async worker concurrency. Default: 25.
<code>--rps RATE</code>	Max requests per second per host. Default: 8.
<code>--proxy URL</code>	HTTP proxy for all outgoing traffic.
<code>--cookie K=V</code>	Cookie. Repeat as needed.
<code>-H, --header K:V</code>	Header. Repeat as needed.
<code>--payloads DIR</code>	Override payload root directory. Default: ./payloads.
<code>--llm / --no-llm</code>	Enable or disable LLM payload generation. Default: enabled.
<code>--evolve / --no-evolve</code>	Enable or disable GA payload evolution. Default: enabled.
<code>--trriage FLOAT</code>	Minimum triage confidence to report. Default: 0.4.
<code>-o, --out DIR</code>	Output directory for reports. Default: ./arachnex-out.
<code>-f, --format FMT</code>	html json md. Repeat for multiple formats.
<code>--save-har</code>	Save full HAR for every verified finding (large).
<code>-q, --quiet</code>	Suppress banner and config printout.
<code>--log-level LEVEL</code>	DEBUG INFO WARNING ERROR. Default: INFO.

7.2 · arachnex dom

Static DOM-XSS audit. Takes one or more JS files or directories and reports candidate source-to-sink taint flows without sending any network traffic.

```
arachnex dom path/to/app/static/js/  
arachnex dom main.bundle.js vendor.js --out dom_audit.txt
```

Output is a table of (file, source, sink, path) tuples. Sources include `location.*`, `document.URL`, `postMessage`, `window.name`, and the storage APIs. Sinks include `innerHTML`, `eval`, `Function()`, `document.write`, `setTimeout` with a string argument, and `jQuery's .html()`.

7.3 · arachnex blind

Run the blind-XSS collaborator server. Listens for callbacks from blind payloads delivered during a scan. See chapter 11 for full infrastructure setup.

```
arachnex blind \  
  --listen 0.0.0.0:8443 \  
  --domain xss.your-collab.com
```

7.4 · arachnex version

Prints the version banner. Useful in CI scripts.

```
arachnex version
```

8 · Scan modes

ArachneX has four scan modes, controlled by `--mode`. Each mode is a coordinated bundle of crawler depth, parameter mining, headless verification, and request budget. You can override any individual setting on top of a mode.

8.1 · quick

Test only the URLs and parameters you explicitly pass. No crawling, no parameter mining. The fastest mode; useful for CI gates against a known endpoint list.

```
arachnex scan -u https://t/path?p=1 --mode quick
```

8.2 · standard

The default. Crawls within scope up to depth 3, tests every discovered parameter, verifies findings in the headless oracle. Good balance of coverage and speed for most engagements.

8.3 · deep

Standard plus parameter mining on each root URL (binary-split bisection over ~75 built-in parameter names plus any wordlist you supply). Adds 15–30% to scan time but commonly surfaces hidden debug or callback parameters.

8.4 · bug-bounty

Deep plus aggressive payload generation: more LLM calls per context, longer GA evolution budgets, more polyglot fallbacks, and active Wayback enrichment when available. This is the slowest mode but the one to use when the goal is to maximize findings on a bug-bounty scope.

NOTE · Mode picker rule of thumb

Use **quick** for CI. Use **standard** when a target is fresh. Use **deep** when standard found nothing or only obvious findings. Use **bug-bounty** when you have a long evening and a scope file with real depth.

9 · Understanding reports

ArachneX writes reports to `./arachnex-out/<scan_id>/`. Three formats are produced when requested with `-f html -f json -f md`.

9.1 · HTML report

Single self-contained file with a dark theme, embedded screenshots (base64), and per-finding cards. Designed for sharing with internal triage teams or attaching to a bug-bounty submission as proof.

9.2 · JSON report

Schema-versioned (`schema_version: 1.0.0`). Stable. This is the file your CI pipeline should consume — never scrape the HTML.

```
{
  "schema": "arachnex/scan-summary",
  "schema_version": "1.0.0",
  "summary": {
    "scan_id": "...",
    "started_at": "2026-06-26T10:00:00Z",
    "findings": [ /* one object per finding */ ],
    "urls_discovered": 42,
    "params_discovered": 117,
    "requests_sent": 4231
  }
}
```

9.3 · Markdown report

Plain Markdown. Paste it into a GitHub issue, Slack channel, PR comment, or a bug-bounty platform's submission form. Includes a summary table and per-finding entries with severity emoji prefixes.

9.4 · Severity and CVSS

Each finding has a severity (info / low / medium / high / critical) and a CVSS 4.0 vector. The triager produces an initial severity from the triage confidence and execution status; CVSS scoring is then applied and may escalate (never downgrade) the severity. The CVSS vector is built from observed attack mechanics (network attack, WAF active or not, authenticated or not, browser exec observed or not, stored or reflected) — see chapter 12 of the architecture document for the full mapping.

10 · The headless oracle

Reflection is not execution. Most public XSS scanners report any response that contains the injected canary; that is why their false-positive rates are 30–70% on real engagements. ArachneX uses a real Chromium browser, driven by Playwright, to confirm that JavaScript actually runs.

10.1 · How it works

Before navigating, ArachneX injects a sentinel script using Playwright's `add_init_script`. This runs before any page script and installs a `window.__arx` object that:

- Generates a per-page random token.
- Re-binds `alert`, `prompt`, and `confirm` to record events.
- Exposes `window.__arx.fired(label, extra)` for explicit signalling.

Three independent signals are then watched:

- **Runtime.consoleAPICalled** — payloads can call `console.log('__arx_exec__ : tag')`.
- **page.on('dialog')** — catches alert/prompt/confirm even if the rebind fails.
- **The __arx sentinel** — catches payloads that don't use a dialog.

Any of the three firing is enough to confirm execution. The triager flips the `executed=True` feature, which is the dominant positive weight in the scoring model.

10.2 · Headless modes

Option	Description
off	No headless verification. Faster, more false positives. Use when scanning endpoints that cannot be rendered (e.g., API-only).
verify	Default. Only candidate findings are run through the oracle. Best balance.
always	Every reflection is verified, even ones with very low triage confidence. Costly; use when you need maximum precision.

10.3 · Performance

The oracle pool is capped at `min(8, --threads)` because each Chromium context uses around 80 MB of RAM. On a 4 GB workstation, eight is the upper bound before swap pressure starts to matter.

11 · Blind XSS infrastructure

Blind XSS fires in internal tools — admin dashboards, log viewers, CRM systems — where the tester cannot see the rendered DOM. The victim browser (usually an internal employee opening your submission) calls home, and that callback is the proof.

11.1 · The collaborator server

ArachneX ships an embedded FastAPI server that listens for callbacks via HTTP (POST), WebSocket, and a pixel-image GET fallback. Run it on a host with a public DNS name pointing to your listener:

```
arachnex blind \  
  --listen 0.0.0.0:8443 \  
  --domain xss.example-collab.com
```

11.2 · Deploying with TLS

Most modern targets only fire blind payloads over HTTPS, so put Caddy or Nginx in front. A minimal Caddy config:

```
xss.example-collab.com {  
    reverse_proxy 127.0.0.1:8443  
}
```

Caddy will issue and renew the certificate automatically. The ArachneX listener itself does not need TLS — terminate it at the reverse proxy.

11.3 · Payload variants delivered

When blind detection is enabled, ArachneX picks payload variants appropriate for each context:

Option	Description
img_onerror_fetch	Standard image-onerror plus fetch with full DOM exfil.
svg onload_fetch	SVG-namespace variant; sometimes survives sanitizers.
event_handler_only	Bare event-handler payload for use inside existing tags.
javascript_scheme	For URL contexts (href, src, formaction).
websocket_loader	Maintains a longer-lived channel for progressive exfil.
dns_exfil	Pure DNS subdomain probe; works behind HTTP-blocking firewalls.

11.4 · Reviewing callbacks

Callbacks are stored in a SQLite database at `arachnex-out/collab.sqlite3`. Each row records the probe ID (which lets ArachneX correlate the callback with the target, injection point, and payload that delivered it), the remote IP, User-Agent, the page URL it fired from, and the cookies (where non-HttpOnly).

```
sqlite3 arachnex-out/collab.sqlite3 \  
  "SELECT probe_id, remote_ip, page_url FROM callbacks ORDER BY ts_ms DESC LIMIT 10;"
```

WARNING · Operational secrecy

Do not host your collaborator on a domain that obviously names you or your client. Use a neutral hostname. Many bug-bounty targets log all outbound network calls; making it obvious which researcher you are reduces the value of the callback.

12 · AI features

ArachneX has three AI-flavoured components. None is required for the tool to operate, and every fallback path is implemented and tested.

12.1 · LLM payload generation

After a reflection is classified into one of seventeen contexts, ArachneX optionally calls Claude with the context label, the byte neighbourhood around the reflection, and the detected WAF vendor (if any). Claude returns a strictly schema-validated JSON array of payloads, each with a rationale and an execution marker. These are tried first, before the static library, on the theory that neighbourhood-aware payloads have a much higher hit rate.

Enable by setting `ANTHROPIC_API_KEY` and passing `--llm` (default on). Disable with `--no-llm`.

12.2 · Genetic-algorithm payload evolution

When a candidate payload is blocked by a WAF (HTTP 403, 406, 419, or a known block signature in the body), ArachneX kicks off a genetic-algorithm search. The grammar is eighteen syntactically-aware transformations: case toggling, comment splitting inside tag names, URL encoding stacks, HTML entity encoding, zero-width joiners inside tags, swapping `aler t` for `top .aler t`, and so on.

Fitness is a weighted blend of three signals — was the request blocked, how intact was the reflection, did the oracle observe execution. Typical convergence is 20–60 generations against production Cloudflare/AWS WAF/Akamai. Disable with `--no-evolve`.

12.3 · Auditable triage scoring

Every finding is scored 0.00–1.00 by a hand-tuned logistic regression over eight features. The weights are visible in `arachnex/ai/triage.py`; the explanation for each finding is written into the report. This is deliberately interpretable rather than a black-box model — in offensive security, a tool you cannot reason about is a tool you cannot trust.

12.4 · LLM cost estimation

A medium-sized scope (a few hundred URLs, maybe a thousand parameter tests) usually costs \$0.10–\$0.50 in Anthropic API spend at default settings. Most of the cost is the per-context payload generation; results are cached by (context, neighborhood-hash, waf), so large scopes do not scale linearly.

13 · Recipes

Reference scans for the situations you will actually encounter. Copy and adapt.

13.1 · Authenticated SPA on a known host

```
arachnex scan -u https://app.example.com/ \
  --mode deep \
  --cookie 'session=eyJhbGciOi...' \
  --cookie 'csrftoken=abc123' \
  -H 'X-CSRF-Token: abc123' \
  --rps 5 \
  -f html -f json
```

13.2 · API endpoint accepting JSON

When the target accepts JSON, give ArachneX a body template with a placeholder; the engine will substitute payloads into the named field:

```
arachnex scan \
  -u https://api.example.com/v1/search \
  -H 'Content-Type: application/json' \
  -H 'Authorization: Bearer eyJh...' \
  --body '{"query": "PAYLOAD", "limit": 10}' \
  --mode deep
```

13.3 · A scope file of multiple hosts

```
# scope.txt
https://app.example.com/
https://api.example.com/v1/
https://admin.example.com/
```

```
arachnex scan --scope scope.txt --mode bug-bounty -f html
```

13.4 · A CI gate that fails on High/Critical findings

```
# .github/workflows/xss.yml (snippet)
- name: Run ArachneX
  run: |
    arachnex scan -u https://staging.example.com/ \
      --mode standard --quiet \
      -f json --out ./out
    python -c "
import orjson, sys
data = orjson.loads(open(next(__import__('pathlib').Path('out').rglob('report.json'))).read())
highs = sum(1 for f in data['summary']['findings'] if f['severity'] in ('high','critical'))
sys.exit(1 if highs else 0)
"
```

13.5 · Investigating a specific suspected parameter

Use quick mode to test one parameter without crawling:

```
arachnex scan \
  -u "https://t.com/search?q=test&category=all" \
  --mode quick \
  --headless always \
```

```
--log-level DEBUG
```

13.6 · Static-only DOM-XSS audit (no traffic)

```
# Download the target's JS bundle first, then:
arachnex dom downloaded/main.bundle.js \
          downloaded/vendor.js
```

13.7 · Routing through Burp Suite

```
arachnex scan -u https://target/ \
  --proxy http://127.0.0.1:8080 \
  --threads 5 --rps 2      # easier on Burp's history
```

13.8 · Blind XSS campaign

Run the collab server (terminal 1), scan with blind payloads enabled (terminal 2):

```
# terminal 1
arachnex blind --listen 0.0.0.0:8443 \
              --domain xss.collab.example.com

# terminal 2
arachnex scan -u https://t.com/submit-form \
  --blind --collab-http https://xss.collab.example.com \
  --mode bug-bounty
```

14 · Troubleshooting

14.1 · "playwright: command not found"

Activate the virtual environment first. If the venv is active and the command still fails, reinstall:

```
pip install -U playwright
playwright install chromium
```

14.2 · "Executable doesn't exist at chromium ..."

Playwright is installed but the browser is not:

```
playwright install chromium
```

On minimal Linux images, also run:

```
playwright install-deps chromium
```

14.3 · SSL: CERTIFICATE_VERIFY_FAILED

Either the target uses a custom CA your trust store doesn't know, or you are routing through Burp/mitmproxy without their CA installed. For a one-off scan, disable verification:

```
ARX_VERIFY_TLS=0 arachnex scan -u https://target/
```

WARNING · Never on production

Disabling TLS verification means you can be MITM'd and fed fabricated responses. Use only for proxied debugging.

14.4 · 429 Too Many Requests

The target's rate limiter has tripped. Reduce request rate:

```
arachnex scan ... --rps 2 --threads 5
```

14.5 · LLM payload generation seems disabled

Check the banner at scan start — if `llm=no` appears, either:

- `ANTHROPIC_API_KEY` is not set in the environment or `.env` file.
- `--no-llm` was passed (explicitly or via shell alias).
- The anthropic Python package is not installed (rerun `pip install`).

14.6 · Scan is too slow

In rough order of impact, try:

- Use `--mode standard` instead of `deep` or `bug-bounty`.
- Use `--headless off` for first-pass triage.
- Increase `--threads` (up to ~50 on a fast workstation).
- Increase `--rps` (only if the target tolerates it).
- Disable evolution: `--no-evolve`.

- Disable LLM payloads: `--no-llm`.

14.7 · "could not start headless oracle"

Playwright failed to launch. The most common causes are missing system libraries on minimal containers and an existing Chromium process holding a profile lock. Run `playwright install-deps chromium`, kill any orphan Chromium processes, and try again. If it still fails, rerun with `--log-level DEBUG` for the Playwright stderr.

14.8 · A finding shows reflection but no execution

This is correct behaviour and not a bug. ArachneX deliberately reports findings only when the headless oracle observes execution. A reflection without execution can be caused by CSP, sandboxing, Trusted Types, server-side encoding, or simply because the reflected bytes never enter an executable context. Lower `--triage` to see lower-confidence reflections as Info findings.

15 · Performance tuning

ArachneX has three concurrency knobs that interact:

15.1 · --threads

The size of the async worker pool. Each worker can have one in-flight HTTP request and zero or one in-flight headless context. Typical values: 10 on a laptop, 25 on a workstation, 50+ on a well-provisioned host. Increasing beyond 50 rarely helps because the per-host rate limit becomes the bottleneck instead.

15.2 · --rps

Requests per second per host, enforced by a token bucket inside `HttpClient`. This is the most important knob for being a good citizen on a bug-bounty scope. Default 8 is deliberately conservative; production targets often tolerate 20–40 RPS without trouble, but ramp up gradually and watch the server's response codes.

15.3 · The headless pool

Capped at $\min(8, \text{threads})$. You cannot raise this from the CLI in v0.1.0 — modify `arachnex/exploitation/headless.py` if you have a beefy host. The 80 MB per-context cost is the limiting factor.

15.4 · A representative budget

Option	Description
Scope	150 URLs, 600 parameters, mode=deep
Threads	25
RPS	10 per host
Headless	verify
LLM	enabled, ~80 calls
Duration	8–12 minutes
Cost	\$0.20 LLM, otherwise free
Findings	5–25 typical, depending on target

16 · Ethics & legal

DANGER · Read this before running anything

ArachneX is a vulnerability discovery tool. Pointing it at a system you do not own and are not explicitly authorized to test is illegal under almost every modern computer-crime statute. The authors accept no liability for misuse.

16.1 · Jurisdictions

If you operate from India: unauthorized scanning is a criminal offence under sections 43 and 66 of the Information Technology Act 2000, with fines up to ₹1 crore and imprisonment up to three years. If you operate from the United States: the Computer Fraud and Abuse Act (18 U.S.C. §1030) applies, with civil and criminal penalties. If you operate from the United Kingdom: the Computer Misuse Act 1990. Most other jurisdictions have equivalent laws.

16.2 · What "authorization" looks like

Authorization, at a minimum, means:

- A written agreement (contract, statement of work, bug-bounty program rules in scope).
- A defined scope: which hosts, which IP ranges, which environments.
- A defined timeframe: from-date and to-date, or open-ended with a notice clause.
- Defined restrictions: rate limits, blackout windows, prohibited techniques.
- A point of contact for incidents and questions.

A verbal "yeah, go ahead" is not authorization. Get it in writing.

16.3 · Bug bounty etiquette

- Read the program scope before pointing ArachneX at anything.
- Honour rate limits aggressively. Use `--rps 2` or lower on first scan.
- Don't store callback data containing user PII any longer than needed for proof.
- Don't pivot from a confirmed XSS into other attacks unless the program scope authorizes it.
- Report responsibly. Give the program time to fix before publishing.

17 · Glossary

Option	Description
Canary	A random short string injected into a parameter to detect reflection. ArachneX uses 'arx' + 6 hex chars.
Context	The syntactic position of a reflection in the response. ArachneX classifies seventeen contexts.
CVSS 4.0	Common Vulnerability Scoring System version 4.0. ArachneX produces vectors and numeric scores for every finding.
DOM XSS	XSS where source-to-sink flow happens entirely client-side via JavaScript, with no server reflection.
Headless oracle	Real Chromium browser used to verify whether a candidate payload actually executes JS.
Injection point	A (location, name) tuple identifying where a payload is injected — e.g. (query, q) or (body, search).
Marker	A greppable token inside a payload that identifies execution — e.g. alert:31337.
mXSS	Mutation XSS. Sanitizers see one set of bytes; the browser re-parses on innerHTML and gets a different DOM.
Oracle	Any component that returns ground truth — here, the headless browser.
Polyglot	A single payload that works across multiple contexts simultaneously.
Reflection	The appearance of an injected canary in a response, anywhere.
Sentinel	Code injected to detect execution — here, the window.__arx object.
Stored XSS	XSS where the payload is persisted server-side and rendered to a different user later.
Triage	Scoring and prioritization of findings. ArachneX uses an auditable logistic model.
WAF	Web Application Firewall. ArachneX fingerprints nine common vendors.

Appendix A · Environment variable reference

Option	Description
ANTHROPIC_API_KEY	Anthropic API key. Required for LLM features.
ARX_LLM_MODEL	Default: claude-sonnet-4-6.
ARX_OUTPUT_DIR	Default report root if --out is not given.
ARX_LOG_LEVEL	DEBUG/INFO/WARNING/ERROR.
ARX_USER_AGENT	Override User-Agent.
ARX_PROXY	HTTP proxy URL.
ARX_VERIFY_TLS	Set to 0 to disable TLS verification (debugging only).
ARX_PAYLOAD_ROOT	Override payload directory location.
ARX_TRIAGE_THRESHOLD	Override default 0.4 triage threshold.
ARX_SAVE_HAR	Set to 1 to save full HARs for verified findings.

Appendix B · Project directory layout

```

arachnex/
├── arachnex/
│   ├── core/
│   │   ├── ai/
│   │   ├── detection/
│   │   ├── discovery/
│   │   ├── exploitation/
│   │   ├── reporting/
│   │   ├── server/
│   │   ├── utils/
│   │   └── cli.py
│   ├── payloads/
│   │   ├── contexts/
│   │   ├── polyglots/
│   │   └── waf_bypass/
│   ├── examples/
│   │   ├── demo_target.py
│   │   └── scope.example.txt
│   ├── tests/
│   │   └── test_smoke.py
│   └── docs/
│       ├── architecture.md
│       └── ai_methodology.md
├── models/
├── pyproject.toml
├── requirements.txt
├── LICENSE
└── README.md
# the Python package
# config, engine, typed models
# LLM, GA, triage, WAF fingerprint
# reflected, DOM, mXSS, blind, context
# crawler, parameter miner
# headless oracle, payload library
# HTML, JSON, Markdown, CVSS
# blind XSS collaborator
# HTTP client, logger
# Typer CLI entry point
# per-context payload files
# cross-context payloads
# vendor-seeded GA starters
# deliberately vulnerable Flask app
# scope file template
# import + roundtrip + classifier tests
# design and rationale
# the Black Hat talk track
# trained model artifacts (empty by default)
```

End of user guide. For architecture and AI-methodology discussion, see `docs/architecture.md` and `docs/ai_methodology.md` in the source distribution.